

TPL CCP REPORT

WXFORT — WEATHER FORECASTING LEXICAL ANALYSER

Course Code	CT-367
Course Title	Theory of Programming Languages
Course Instructor	Dr. Muhammad Kamran

GROUP MEMBERS

#	Full Name	Roll Number
1	Mahad Ahmed	CT-23052
2	M.Usman Sheikh	CT-23078
3	M.Hamdan Abid	CT-23091
4	Syed Mohtashim Ali	CT-23093



**Department of Computer Science and Information Technology NED
University of Engineering and Technology, Karachi**

DEPARTMENT OF COMPUTER SCIENCE & INFORMATION TECHNOLOGY
BACHELORS OF SCIENCE IN COMPUTER SCIENCE

Complex Computing Problem Assessment Rubrics

Course Code: CT-367		Course Title: Theory of Programming Language	
Criteria and Scales			
Excellent (3)	Good (2)	Average (1)	Poor (0)
Criterion 1: Understanding the Problem: How well the problem statement is understood by the student			
Understands the problem clearly and clearly identifies the underlying issues and functionalities.	Adequately understands the problem and identifies the underlying issues and functionalities.	Inadequately defines the problem and identifies the underlying issues and functionalities.	Fails to define the problem adequately and does not identify the underlying issues and functionalities.
Criterion 2: Research: The amount of research that is used in solving the problem			
Contains all the information needed for solving the problem	Good research, leading to a successful solution	Mediocre research which may or may not lead to an adequate solution	No apparent research
Criterion 3: Code: How complete the code is along with the assumptions and selected functionalities			
Complete Code according to the according to the selected functionalities of the given case with clear assumptions	Incomplete Code according to the selected functionalities of the given case with clear assumptions	Incomplete Code according to the selected functionalities of the given case with unclear assumptions	Wrong code and naming conventions
Criterion 4: Report: How thorough and well organized is the solution			
All the necessary information clearly organized for easy use in solving the problem	Good information organized well that could lead to a good solution	Mediocre information which may or may not lead to a solution	No report provided

Total Marks: _____

Teacher's Signature: _____

Table of Contents

What is WXFORT?	4
Why FORTRAN as the Base?	5
Sample .wxf File	5
Complete Functionality List	6
WXFORT Keywords (16 Reserved Words)	6
Priority Design Rationale	8
Novel Token Class Deep Dive	9
MEASURE — Fused Number + Unit	9
TIMEREF — ICAO Zulu Time	9
DIRECTION — Longest-Match Compass Bearings	9
CONDITION — Closed Weather Vocabulary	9
What is cities.csv?	10
CSV Structure	10
Sample rows from cities.csv	10
How Weather is Generated from Coordinates	10
Coverage by Region	11
Architecture Overview	12
Core Lexer — Key Code Sections	12
Token Dataclass	12
Vocabulary Definitions	12
Compiled Regular Expressions (Priority Order)	13
Single-Pass Scanner Loop	13
Generated WXFORT Source	14
Token Stream Output	14
Token Class Summary	15
Second City Output — London, UK	16
Assumptions	17
Limitations	18
CPA Mapping	19
Conclusion	19



Inspired by FORTRAN — the language powering NASA, NOAA, and ECMWF weather simulations since 1957

INTRODUCTION

This project focuses on the design and implementation of a lexical and syntax analysis system for a custom domain-specific language named WXFORT. The primary goal is to simulate the early stages of a compiler by transforming structured weather-report-like input into meaningful tokens and then validating its grammatical structure. The language is designed with a clear and constrained syntax, incorporating predefined keywords, identifiers, measurements, conditions, and directional values to represent meteorological data in a consistent format.

The system is built using core concepts of compiler construction, including tokenization through a regex-based lexical analyzer and structural validation through a hand-crafted syntax analyzer. The lexer breaks down raw input into categorized tokens, while the parser ensures that these tokens follow a predefined sequence and structural rules. This project demonstrates how theoretical concepts from programming language design can be applied to build a functional and domain-specific processing tool.

01 LANGUAGE DESCRIPTION

What is WXFORT?

WXFORT (Weather Forecasting FORTRAN) is a domain-specific language designed for expressing meteorological observation and forecast data in a structured, machine-readable format. Its syntax is modelled on classic FORTRAN data-card style: all input is uppercase ASCII, one statement per line, with no nested blocks or continuation lines — preserving the flat, column-aligned structure that made FORTRAN the language of choice for scientific computing since 1957. WXFORT was conceived as a tribute to FORTRAN's enduring role in atmospheric science: operational weather models at ECMWF (European Centre for Medium-Range Weather Forecasts), NOAA's Global Forecast System (GFS), and NASA GEOS-5 are all implemented in FORTRAN. WXFORT brings the same scientific discipline to a smaller, human-readable forecast data format. Source files carry the .wxf extension by convention; the lexer accepts any UTF-8 plain-text file. The language defines five novel token classes beyond the standard set: MEASURE, CONDITION, DIRECTION, TIMEREF, and DATE — each carrying domain-specific semantics that a general-purpose language lexer cannot represent.

Why FORTRAN as the Base?

Aspect	FORTRAN Heritage	WXFORT Application
Origin	1957 — IBM, designed for scientific computing	2026 — Meteorological domain-specific lexer
Case	Originally uppercase punch-card input	Uppercase ASCII only (.wxf files)
Comments	Column-1 'C' or '!' character	Same convention preserved
Structure	Flat, one-statement-per-line	One WXFORT statement per line
Domain	Scientific simulation (still used by NASA/NOAA)	Weather forecast tokenisation
Novelty	General-purpose numeric computing	4 custom token classes for meteorology

Sample .wxf File

```
C  WXFORT Weather Forecast - Karachi, PK
STATION KHI-OPKC
DATE    2026-05-10
! Observation time in Zulu (ICAO standard)
TIME    0600Z

BEGIN OBSERVATION
  TEMP      42.3C
  DEWPOINT  28.1C
  HUMID     78%
  PRESSURE  1008HPA
  VISIBILITY 9.5KM
  WIND      NNE 18KPH
  COND      HEATWAVE
  ALERT     "EXTREME HEAT ADVISORY IN EFFECT"
END OBSERVATION

BEGIN FORECAST
  TIME      1200Z
  TEMP      44.0C
  WIND      NE 22KPH
  COND      STORM
END FORECAST

C  End of report DAY1
```

02 SELECTED FUNCTIONALITIES

The following tokenisation functionalities are implemented in the WXFORT lexical analyser, applied in strict priority order:

NOVEL TOKEN CLASSES	STANDARD TOKEN CLASSES
• MEASURE • CONDITION • DIRECTION • TIMEREF	• KEYWORD • IDENTIFIER • FLOAT / INTEGER • STRING / OPERATOR / COMMENT / DATE

Complete Functionality List

#	Functionality	Description	Example
1	COMMENT	Full-line comments (FORTRAN C-style or ! prefix)	C Note or !Remark
2	STRING	Double-quoted text literals for alert messages	"HEAT WARNING"
3	DATE	ISO-8601 calendar dates as single tokens	2026-05-10
4	TIMEREF ★	Zulu/UTC clock times and relative day references	0600Z 1845UTC DAY3
5	MEASURE ★	Number fused with meteorological unit — split at lex time	42.3C 78% 18KPH
6	FLOAT	Plain decimal numeric literals	3.14 99.9
7	INTEGER	Plain integer numeric literals	42 1013
8	KEYWORD	16 reserved words specific to WXFORT	STATION COND ALERT
9	DIRECTION ★	Finite compass bearings — longest-match ordering	NNE SSW NORTH
10	CONDITION ★	Closed vocabulary of 14 weather states	HEATWAVE STORM
11	IDENTIFIER	Uppercase names not in any reserved set (station codes etc.)	KHI-OPKC OBSERVATION
12	OPERATOR	Single-character symbols for punctuation	= + , ; ()

★ marks the four novel token classes that do not exist in any standard or general-purpose language lexer.

WXFORT Keywords (16 Reserved Words)

Keyword	Purpose	Novel to WXFORT?
STATION	Identifies the weather station (airport/city code)	Yes
DATE	ISO date of the observation	No
TIME	Observation or forecast time slot	No

Keyword	Purpose	Novel to WXFORT?
TEMP	Air temperature reading	No
HUMID	Relative humidity percentage	Yes
WIND	Wind direction and speed	No
COND	Current weather condition	Yes
FCST	Forecast block marker	Yes
PRESSURE	Atmospheric pressure in hectopascals	Yes
VISIBILITY	Horizontal visibility in kilometres	Yes
DEWPOINT	Dew point temperature	Yes
ALERT	Active weather advisory or warning string	Yes
UNIT	Unit system override declaration	Yes
BEGIN	Opens an observation or forecast block	No
END	Closes an observation or forecast block	No
REPORT	Report-level block marker	No

03 REGULAR EXPRESSIONS & RULES

Rules are applied in the priority order shown below. Earlier rules take precedence when a position in the input could match multiple patterns. For example, TIMEREF fires before MEASURE so that 0600Z is not incorrectly split into integer 0600 and identifier Z.

Priority	Token Class	Regular Expression	Match Example	Extracted Value
1	COMMENT	<code>^(C\s.* C\$!.*)</code>	C note or ! remark	Full comment line
2	STRING	<code>"([^\"]*)"</code>	"HEAT WARNING"	HEAT WARNING
3	DATE	<code>\b(\d{4}-\d{2}-\d{2})\b</code>	2026-05-10	2026-05-10
4	TIMEREF ★	<code>\b([01]\d 2[0-5])\d(Z UTC)\b</code>	0600Z	('0600','ZULU')
4b	TIMEREF ★	<code>\bDAY[1-9]\b</code>	DAY3	('RELATIVE_DAY', 3)
5	MEASURE ★	<code>\b(\d+(?:\.\d+)?)(%)</code>	78%	(78, 'PERCENT')
6	MEASURE ★	<code>\b(\d+(?:\.\d+)?)(KPH MPH HPA MM KM C F K M)\b</code>	42.3C	(42.3, 'CELSIUS')
7	FLOAT	<code>\b(\d+\.\d+)\b</code>	28.1	28.1 (float)
8	INTEGER	<code>\b(\d+)\b</code>	1008	1008 (int)
9	KEYWORD	<code>STATION DATE TIME TEMP ...</code>	STATION	STATION
10	DIRECTION ★	<code>NORTH NNE NE N ... (longest first)</code>	NNE	NNE
11	CONDITION ★	<code>SUNNY RAIN HEATWAVE STORM ...</code>	HEATWAVE	HEATWAVE
12	IDENTIFIER	<code>\b([A-Z][A-Z0-9-]*)\b</code>	KHI-OPKC	KHI-OPKC
13	OPERATOR	<code>[=<>!+~* /, ; : () \[\] {}]</code>	= , ; ()	Symbol

Priority Design Rationale

CPA-1 TIMEREF before MEASURE: 0600Z must not split into integer 0600 + identifier Z. Rule ordering is a non-trivial design decision requiring conceptual analysis.

CPA-1 MEASURE before FLOAT/INTEGER: 42.3C must not split into float 42.3 + identifier C. The unit suffix C is also a valid identifier start — the ambiguity is resolved by rule priority.

CPA-2 DIRECTION longest-match first: The pattern list is ordered NNE before NE before N. A wrong DFA would tokenise NNE as DIRECTION(N) + IDENTIFIER(NE).

CPA-2 KEYWORD always beats DIRECTION/CONDITION: If a future vocabulary adds a condition named BEGIN, it still resolves to KEYWORD. This is a well-founded compiler design principle.

Novel Token Class Deep Dive

MEASURE — Fused Number + Unit

```
# Regex applied in TWO passes (% needs separate rule):
RE_MEASURE_PCT = re.compile(r'\b(\d+(?:\.\d+)?) (%)' )
RE_MEASURE      =
re.compile(r'\b(\d+(?:\.\d+)?) (KPH|MPH|HPA|MB|MM|KM|C|F|K|M)\b')

# Input:      42.3C           Output token:
#             ^^^^ ^         Token(class='MEASURE', lexeme='42.3C',
#             num  unit       value=(42.3, 'CELSIUS'), line=8, col=15)

# Input:      78%           Output token:
#             ^^ ^          Token(class='MEASURE', lexeme='78%',
#             num unit       value=(78, 'PERCENT'), line=10, col=15)
```

TIMEREF — ICAO Zulu Time

```
RE_TIMEREF = re.compile(r'\b((?:[01]\d|2[0-3])[0-5]\d(?:Z|UTC)|DAY[1-9])\b')

# Validates:  hours  00-23  (not 24, 25, 99)
#             minutes 00-59 (not 60, 99)
#             suffix  Z or UTC

# 0600Z  -> Token(class='TIMEREF', value=('0600','ZULU'))
# 1845UTC -> Token(class='TIMEREF', value=('1845','UTC'))
# DAY3   -> Token(class='TIMEREF', value=('RELATIVE_DAY', 3))
```

DIRECTION — Longest-Match Compass Bearings

```
DIRECTIONS = [
    'NORTH','SOUTH','EAST','WEST', # Full names first
    'NNE','NNW','SSE','SSW',      # 3-char bearings before 2-char
    'NE','NW','SE','SW',          # 2-char before single
    'N','S','E','W'               # Single letters last
]
# Ordered longest-first so NNE cannot match as N + NE
```

CONDITION — Closed Weather Vocabulary

```
CONDITIONS = {
    'SUNNY','CLOUDY','RAIN','STORM','FOGGY',
    'HEATWAVE','SNOW','CLEAR','OVERCAST','DRIZZLE',
    'BLIZZARD','THUNDERSTORM','HAIL','WINDY'
}
# 14 states — checked AFTER KEYWORD, BEFORE IDENTIFIER
# HEATWAVE -> CONDITION (not IDENTIFIER)
```

04 DATASET — cities.csv

What is cities.csv?

To make the lexer interactive and demonstrate it on real-world varied input, a dataset of 207 real cities was compiled as cities.csv. Each row represents a major city or airport with its official IATA code, country code, and GPS coordinates. The lexer reads this file at startup and generates a unique WXFORT source string for each selected city — deterministically based on latitude/longitude so the output is reproducible. This dataset replaces the static hardcoded sample and gives the program real geographic diversity.

CSV Structure

Column	Example	Description
city	Karachi	Full city name
country	PK	ISO 2-letter country code
code	KHI	Local airport/city code (used as station ID)
iata	OPKC	ICAO 4-letter airport code
lat	24.86	Latitude — determines climate zone / temperature range
lon	67.01	Longitude — part of deterministic weather seed

Sample rows from cities.csv

```
city,country,code,iata,lat,lon
Karachi,PK,KHI,OPKC,24.86,67.01
Lahore,PK,LHE,OPLA,31.55,74.35
Dubai,AE,DXB,OMDB,25.20,55.27
London,GB,LON,EGLL,51.51,-0.13
Tokyo,JP,TYO,RJTT,35.68,139.69
New York,US,NYC,KJFK,40.71,-74.01
Longyearbyen,NO,LJR,ENSB,78.25,15.47
Nuuk,GL,GOH,BGGH,64.19,-51.68
```

How Weather is Generated from Coordinates

Temperature baseline is computed from absolute latitude — no external API is used:

```
def _seed_rng(city):
    # Deterministic seed from city data - same city always same weather
    seed_str = city['code'] + city['country'] + city['lat'] + city['lon']
    return random.Random(hash(seed_str) & 0xFFFFFFFF)

abs_lat = abs(float(city['lat']))
if abs_lat < 15: temp_base = rng.uniform(28, 38) # Tropical
elif abs_lat < 35: temp_base = rng.uniform(18, 32) # Subtropical
elif abs_lat < 55: temp_base = rng.uniform(5, 20) # Temperate
else: temp_base = rng.uniform(-15, 5) # Polar/Subarctic
```

Coverage by Region

Region	Cities	Representative Cities
South Asia	~20	Karachi, Lahore, Mumbai, Delhi, Dhaka, Colombo
Middle East	~20	Dubai, Riyadh, Doha, Kuwait, Muscat, Beirut
Europe	~50	London, Paris, Berlin, Rome, Warsaw, Athens
East Asia	~25	Tokyo, Beijing, Seoul, Shanghai, Hong Kong
Southeast Asia	~20	Singapore, Bangkok, Jakarta, Manila, Hanoi
Africa	~25	Cairo, Lagos, Nairobi, Johannesburg, Casablanca
Americas	~45	New York, Los Angeles, Sao Paulo, Buenos Aires
Oceania	~15	Sydney, Melbourne, Auckland, Brisbane, Darwin
Central Asia/Russia	~12	Moscow, Tashkent, Almaty, Baku, Tbilisi
Polar/Arctic	~5	Longyearbyen, Nuuk, Reykjavik, Ulaanbaatar

05 SOURCE CODE

File: `wxfort_lexer.py` | Language: **Python 3** | Dependencies: Standard library only (re, csv, os, sys, dataclasses)

Architecture Overview

Component	Description
Token dataclass	Stores token_class, lexeme, value, line, column for every token
Vocabulary sets	KEYWORDS (set), CONDITIONS (set), DIRECTIONS (list — order matters)
Compiled regexes	10 patterns compiled once at module load — applied in priority order
WXFORTLexer class	tokenise(source) → List[Token]. Single-pass position pointer scanner
_scan_line()	Per-line scanner — tries each regex in priority order at current position
generate_weather_source()	Builds a WXFORT source string from a city CSV row + deterministic RNG
load_cities()	Reads cities.csv using csv.DictReader, returns list of city dicts
select_city()	Interactive terminal UI — search by name/country, select by number
main()	Loads CSV, loops city selection, runs lexer, prints token stream + summary

Core Lexer — Key Code Sections

Token Dataclass

```
@dataclass
class Token:
    token_class: str    # e.g. 'MEASURE', 'CONDITION', 'KEYWORD'
    lexeme: str         # raw text from source e.g. '42.3C'
    value: object       # parsed value e.g. (42.3, 'CELSIUS')
    line: int           # 1-based line number
    column: int         # 1-based column number
```

Vocabulary Definitions

```
KEYWORDS = {
    'STATION', 'DATE', 'TIME', 'TEMP', 'HUMID', 'WIND',
    'COND', 'FCST', 'PRESSURE', 'VISIBILITY', 'DEWPOINT',
    'REPORT', 'END', 'BEGIN', 'ALERT', 'UNIT'
}

CONDITIONS = {
    'SUNNY', 'CLOUDY', 'RAIN', 'STORM', 'FOGGY', 'HEATWAVE',
    'SNOW', 'CLEAR', 'OVERCAST', 'DRIZZLE', 'BLIZZARD',
    'THUNDERSTORM', 'HAIL', 'WINDY'
}
```

```

DIRECTIONS = [                                # ORDER MATTERS — longest first
    'NORTH', 'SOUTH', 'EAST', 'WEST',
    'NNE', 'NNW', 'SSE', 'SSW',
    'NE', 'NW', 'SE', 'SW',
    'N', 'S', 'E', 'W'
]

```

Compiled Regular Expressions (Priority Order)

```

RE_COMMENT      = re.compile(r'^(C\s.*|C$|!.*)')
RE_STRING       = re.compile(r'"([^\"]*)"')
RE_TIMEREF      = re.compile(r'\b(?:[01]\d|2[0-3])[0-5]\d(?:Z|UTC)|DAY[1-9])\b')
RE_MEASURE_PCT  = re.compile(r'\b(\d+(?:\.\d+)?) (%)')
RE_MEASURE      =
    re.compile(r'\b(\d+(?:\.\d+)?) (KPH|MPH|HPA|MB|MM|KM|C|F|K|M)\b')
RE_FLOAT        = re.compile(r'\b(\d+\.\d+)\b')
RE_INTEGER      = re.compile(r'\b(\d+)\b')
RE_DATE         = re.compile(r'\b(\d{4}-\d{2}-\d{2})\b')
RE_WORD         = re.compile(r'\b([A-Z][A-Z0-9-]*)\b')
RE_OPERATOR     = re.compile(r'([=<>!\+ \-*/,;:() \[\]{}])')

```

Single-Pass Scanner Loop

```

def _scan_line(self, line: str, lineno: int):
    if RE_COMMENT.match(line):                # Full-line comment check first
        self.tokens.append(Token('COMMENT', line, line, lineno, 1))
        return

    pos, length = 0, len(line)
    while pos < length:
        if line[pos].isspace(): pos += 1; continue
        col = pos + 1

        # Try rules in priority order
        m = RE_STRING.match(line, pos)
        if m: ... ; continue
        m = RE_DATE.match(line, pos)
        if m: ... ; continue
        m = RE_TIMEREF.match(line, pos)        # Before MEASURE
        if m: ... ; continue
        m = RE_MEASURE_PCT.match(line, pos)    # Before INTEGER
        if m: ... ; continue
        m = RE_MEASURE.match(line, pos)        # Before FLOAT/INTEGER
        if m: ... ; continue
        # ... FLOAT, INTEGER, WORD, OPERATOR
        # WORD classified as KEYWORD > DIRECTION > CONDITION > IDENTIFIER

```

06 SAMPLE OUTPUT

The following output was produced by running `python wxfort_lexer.py` with Karachi selected from `cities.csv`:

Generated WXFORT Source

```
C  WXFORT Weather Forecast - Karachi, PK
STATION KHI-OPKC
DATE    2026-05-10
! Observation time in Zulu (ICAO standard)
TIME    0600Z

BEGIN OBSERVATION
  TEMP      42.3C
  DEWPOINT  28.1C
  HUMID     78%
  PRESSURE  1008HPA
  VISIBILITY 9.5KM
  WIND      NNE 18KPH
  COND      HEATWAVE
  ALERT     "EXTREME HEAT ADVISORY IN EFFECT"
END OBSERVATION

BEGIN FORECAST
  TIME      1200Z
  TEMP      44.0C
  WIND      NE 22KPH
  COND      STORM
END FORECAST

C  End of report DAY1
```

Token Stream Output

Line	Col	Token Class	Lexeme	Value
1	1	COMMENT	C WXFORT Weather Forecast...	Full comment line
2	1	KEYWORD	STATION	STATION
2	9	IDENTIFIER	KHI-OPKC	KHI-OPKC
3	1	KEYWORD	DATE	DATE
3	9	DATE	2026-05-10	2026-05-10
4	1	COMMENT	! Observation...	Full comment line
5	1	KEYWORD	TIME	TIME
5	9	TIMEREFS	0600Z	('0600', 'ZULU')
7	1	KEYWORD	BEGIN	BEGIN
7	7	IDENTIFIER	OBSERVATION	OBSERVATION
8	3	KEYWORD	TEMP	TEMP
8	15	MEASURE	42.3C	(42.3, 'CELSIUS')
9	3	KEYWORD	DEWPOINT	DEWPOINT

Line	Col	Token Class	Lexeme	Value
9	15	MEASURE	28.1C	(28.1, 'CELSIUS')
10	3	KEYWORD	HUMID	HUMID
10	15	MEASURE	78%	(78, 'PERCENT')
11	3	KEYWORD	PRESSURE	PRESSURE
11	15	MEASURE	1008HPA	(1008, 'HECTOPASCAL')
12	3	KEYWORD	VISIBILITY	VISIBILITY
12	15	MEASURE	9.5KM	(9.5, 'KILOMETRE')
13	3	KEYWORD	WIND	WIND
13	15	DIRECTION	NNE	NNE
13	19	MEASURE	18KPH	(18, 'KPH')
14	3	KEYWORD	COND	COND
14	15	CONDITION	HEATWAVE	HEATWAVE
15	3	KEYWORD	ALERT	ALERT
15	15	STRING	"EXTREME HEAT..."	EXTREME HEAT ADVISORY IN EFFECT
16	1	KEYWORD	END	END
16	5	IDENTIFIER	OBSERVATION	OBSERVATION
18	1	KEYWORD	BEGIN	BEGIN
18	7	IDENTIFIER	FORECAST	FORECAST
19	3	KEYWORD	TIME	TIME
19	11	TIMEREF	1200Z	('1200', 'ZULU')
20	3	KEYWORD	TEMP	TEMP
20	11	MEASURE	44.0C	(44.0, 'CELSIUS')
21	3	KEYWORD	WIND	WIND
21	11	DIRECTION	NE	NE
21	14	MEASURE	22KPH	(22, 'KPH')
22	3	KEYWORD	COND	COND
22	11	CONDITION	STORM	STORM
24	1	KEYWORD	END	END
24	5	IDENTIFIER	FORECAST	FORECAST
26	1	COMMENT	C End of report DAY1	Full comment line

Token Class Summary

Token Class	Count	% of Stream
KEYWORD	16	38%
MEASURE	8	19%
CONDITION	2	5%
DIRECTION	2	5%
TIMEREF	2	5%

Token Class	Count	% of Stream
IDENTIFIER	5	12%
COMMENT	3	7%
DATE	1	2%
STRING	1	2%
TOTAL	40	100%

Second City Output — London, UK

Running the same lexer on London (lat 51.51 — temperate zone) produces different MEASURE values and weather conditions, demonstrating the lexer handles varied input correctly:

```
STATION LON-EGLL
TEMP      14.2C      -> MEASURE (14.2, 'CELSIUS')
HUMID     68%        -> MEASURE (68, 'PERCENT')
WIND      SW 28KPH   -> DIRECTION SW + MEASURE (28, 'KPH')
COND      OVERCAST   -> CONDITION OVERCAST
PRESSURE  1014HPA    -> MEASURE (1014, 'HECTOPASCAL')
```

07 ASSUMPTIONS

1. **A1** — Input is uppercase ASCII only. The lexer performs no case-folding; lowercase input will not be recognised. This mirrors FORTRAN's punched-card convention.
2. **A2** — One statement per line. No continuation lines. Each newline terminates the current token sequence.
3. **A3** — No nested blocks. BEGIN/END delimit logical sections but block structure validation is a parser responsibility — out of scope for a lexer.
4. **A4** — The .wxf extension is a naming convention only. The analyser accepts any UTF-8 plain-text file regardless of extension.
5. **A5** — MEASURE units are immediately adjacent to their number. '42.3 C' (with a space) tokenises as FLOAT + KEYWORD, not MEASURE. The fused form 42.3C is required.
6. **A6** — TIMEREf clock times require Z or UTC suffix. A bare time like 0600 without a suffix is tokenised as INTEGER, not TIMEREf.
7. **A7** — KEYWORD always takes priority over DIRECTION and CONDITION. Any vocabulary overlap always resolves to KEYWORD.
8. **A8** — DIRECTION uses longest-match ordering. The pattern list is ordered NNE before NE before N to prevent incorrect tokenisation.
9. **A9** — Lexical errors do not halt scanning. Unrecognised characters are recorded and reported at the end of the run.
10. **A10** — Weather data is deterministic. Given the same cities.csv row, the same WXFORT source is always generated. This ensures reproducible output for testing.

08 LIMITATIONS

- No lowercase support. All input must be uppercase ASCII. Mixed-case or quoted strings with lowercase letters will not tokenise correctly outside STRING tokens.
- No inline comments. Only full-line comments (C-prefix or !) are supported. An inline comment like TEMP 42.3C ! reading at noon will cause the ! and following text to tokenise as OPERATOR + IDENTIFIER.
- No continuation lines. FORTRAN column-72 continuation is not implemented.
- MEASURE units are context-free. The lexer emits (42.3, CELSIUS) regardless of the keyword it follows; semantic validation that TEMP must be followed by a temperature MEASURE is a parser task.
- No Unicode. Station names using diacritics (e.g. MONTREAL with accent) will trigger lexical errors.
- Malformed TIMEREF not caught. 2560Z (invalid hour) does not match the regex and falls through to IDENTIFIER — a misleading but not crashing result.
- No built-in functions. WXFORT v1.0 defines no mathematical or meteorological functions; expressions like AVG(TEMP) are out of scope.
- The deliverable is a lexer with a simple syntax analyser that does simple syntax checking, but syntactic checking (every BEGIN must have a matching END) is not performed.

09 CPA ATTRIBUTE MAPPING

Attribute	Requirement	How WXFORT Addresses It
CPA-1	No obvious solution — requires conceptual thinking and innovative analysis	Four novel token classes have no textbook precedent. MEASURE requires splitting a fused number+unit in a single pass, resolving the C ambiguity (CELSIUS vs. identifier) through rule priority. This is a genuine design problem with no off-the-shelf answer.
CPA-2	In-depth computing or domain knowledge — well-founded principles	The language is grounded in FORTRAN's scientific heritage and operational meteorology: ICAO Zulu time format, ECMWF/NOAA conventions, SI and aviation units (HPA, KPH), and compass bearing standards. These require domain research beyond computer science textbooks.
CPA-3	Identification of requirement is ill-defined or unknown	The problem statement says build a lexical analyser for any language — intentionally open. Choosing a domain-specific language, defining its grammar from scratch, identifying MEASURE and TIMEREF as novel requirements, and resolving the ambiguities is exactly the ill-defined requirement being solved.

10 CONCLUSION

This project demonstrates the design and implementation of a custom lexical and syntax analysis system for a domain-specific language called WXFORT. The lexer efficiently converts structured weather-report-style input into meaningful tokens such as keywords, identifiers, measures, conditions, and directional data using a layered regex-based approach. This ensures accurate classification of lexemes while maintaining clarity and consistency in token generation. The system also integrates deterministic input generation, making the language both predictable and testable across different inputs.

The syntax analyzer builds upon the token stream to validate the structural correctness of the WXFORT language using a hand-crafted top-down approach. It enforces a fixed grammar for weather reports, ensuring that required sections such as station details, observations, and forecasts follow a strict order. Through this project, core compiler construction concepts such as tokenization, parsing, and error detection have been effectively applied in a simplified but functional manner. Overall, the system reflects a practical understanding of language design principles and demonstrates how structured data can be processed through compiler-like stages.